



Zoho Schools for Graduate Studies



Notes

ARRAYS

1)What are arrays?

Arrays are object that can store multiple elements or items or datum of similar data type in contiguous (or continuous) memory location.

2)How many ways you can declare an array?

There are 2 ways to declare an array.

3)How to declare an array?

1. `int[] a;` //java style of declaring an array
2. `int b[];` // c/c++ style of declaring an array

4) Legal ways to declare a 2-D array:

```
int[][] arr; // recommended way to declare a 2-D array
int [] c [];
int [] [] d;
int e [] [];
```

But these ways are not recommended.

5)Can we specify the size of array at the time of declaration?

No, we can never specify the size of array at the place of declaration.

```
//int[5] a;
//Gives compiler error
```

No memory will be allocated at the time of declaration.

Size of an array should be specified only at the time of creating or constructing or initializing the array.

6) Creating or constructing or initializing the array:

1.By using “new” keyword

```
int[] a = new int[5];
```

If we are not giving the size of the array means it gives compiler error.

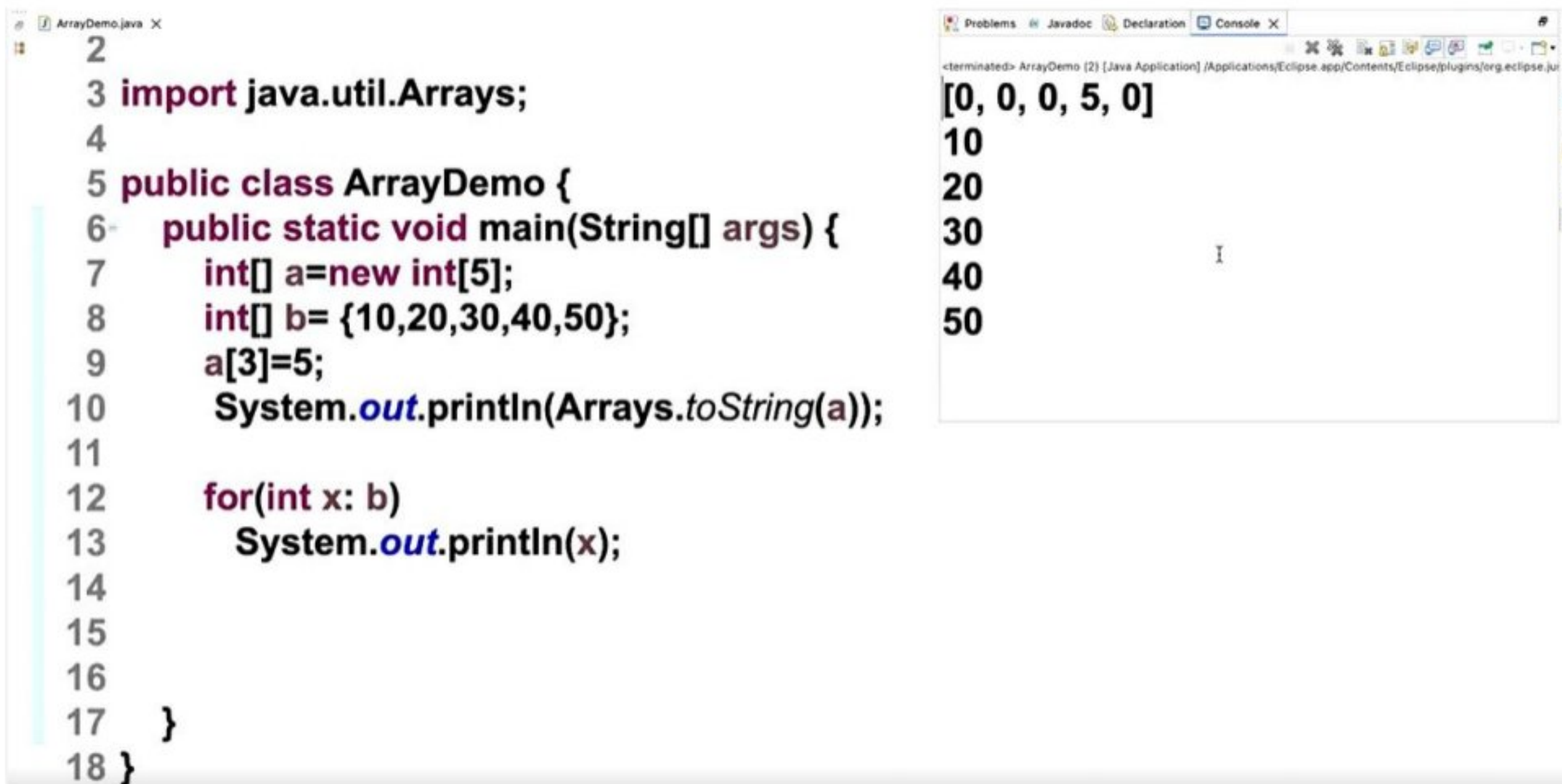
```
1 ArrayDemo.java X
2
3 public class ArrayDemo {
4     public static void main(String[] args) {
5         int[] a=new int[5];
6         System.out.println(a);
7
8     }
9 }
```

```
Problems Javadoc Declaration Console X
<terminated> ArrayDemo (2) [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plugins/org.eclipse.j
[!@64616ca2
```

2.Directly constructing the array or initializing the array

```
int[] b = {10,20,30,40,50};
```

Default initialization will apply even if the array is partially initialized.

The screenshot shows the Eclipse IDE with two windows. The left window, titled 'ArrayDemo.java', contains the following Java code:

```
2
3 import java.util.Arrays;
4
5 public class ArrayDemo {
6     public static void main(String[] args) {
7         int[] a=new int[5];
8         int[] b= {10,20,30,40,50};
9         a[3]=5;
10        System.out.println(Arrays.toString(a));
11
12        for(int x: b)
13            System.out.println(x);
14
15
16
17    }
18 }
```

The right window, titled 'Console', shows the output of the program:

```
<terminated> ArrayDemo (2) [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plugins/org.eclipse.jdt
[0, 0, 0, 5, 0]
10
20
30
40
50
```

3.Initialising the array at the place of construction

```
int[] c = new int[]{1,2,3,4,5};
```

Note:

In second case,

```
int[] b;
b = {10,20,30,40,50};
```

We cannot do initialization and declaration separately. It gives compiler error.

In first case,

```
int[] a = new int[5];
a={1,2,3,4,5};
//Compiler error
```

*Arrays can be initialized directly only at the place of declaration.

In third case, can we specify the size of array?

Eg: `int[] c = new int[7]{1,2,3,4,5,6,7};`

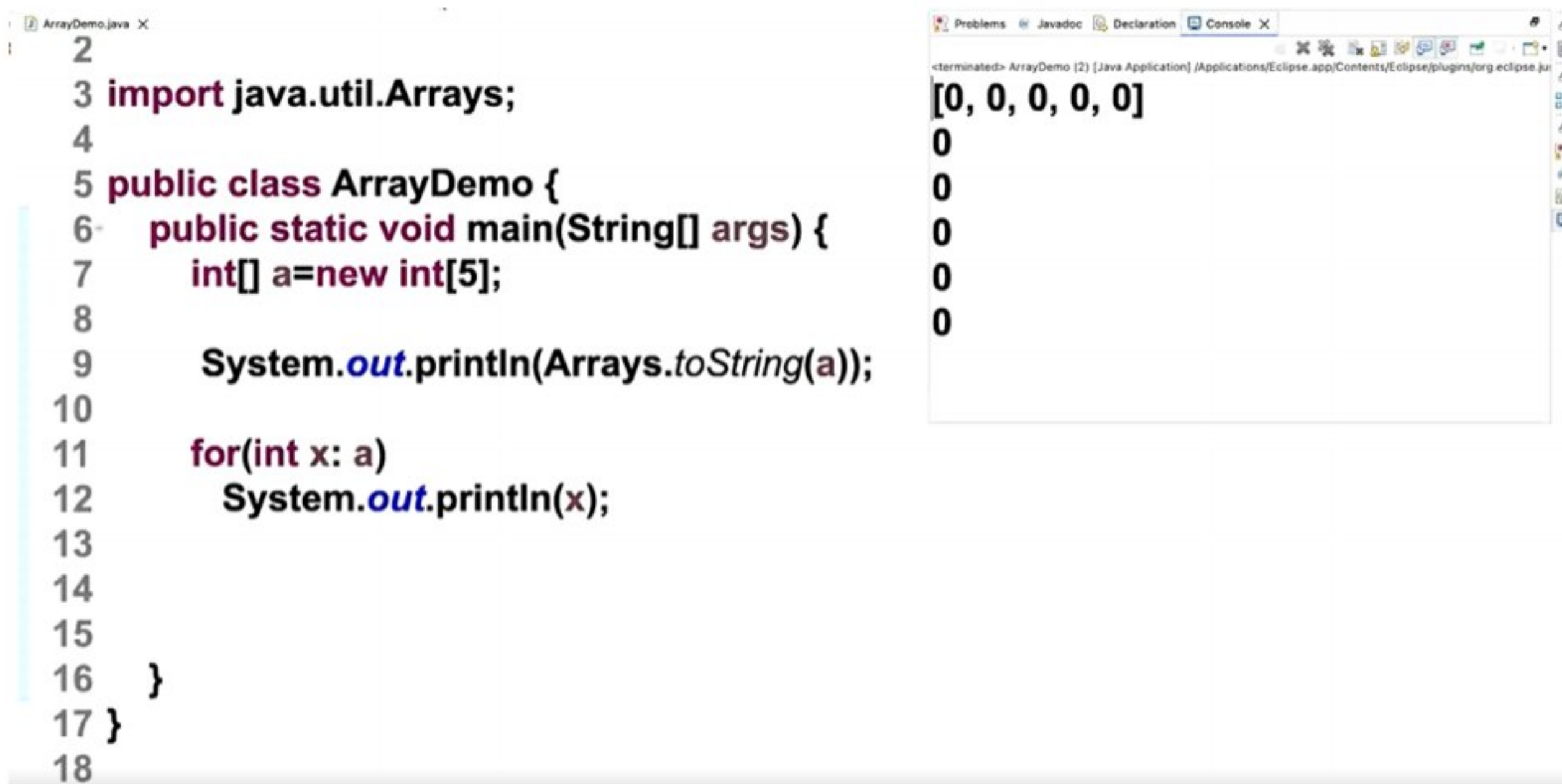
No, we cannot specify the size of the array here.

The size of the array must not be specified when the array is

initialized at the place of construction or instantiation or construction of an array.

7)How can we access the array elements?

- 1.Enhanced for loop or for each loop
- 2.Arrays.toString(); (present in utility package)



The screenshot shows the Eclipse IDE with a Java file named `ArrayDemo.java` open. The code defines a class `ArrayDemo` with a `main` method. Inside `main`, an integer array `a` of size 5 is created and initialized with default values (0s). The code then prints the array using `Arrays.toString(a)` and iterates over each element using an enhanced for loop, printing each element. The console output on the right shows the array `[0, 0, 0, 0, 0]` and each element `0` on a new line.

```
2
3 import java.util.Arrays;
4
5 public class ArrayDemo {
6     public static void main(String[] args) {
7         int[] a=new int[5];
8
9         System.out.println(Arrays.toString(a));
10
11        for(int x: a)
12            System.out.println(x);
13
14
15
16    }
17 }
18
```

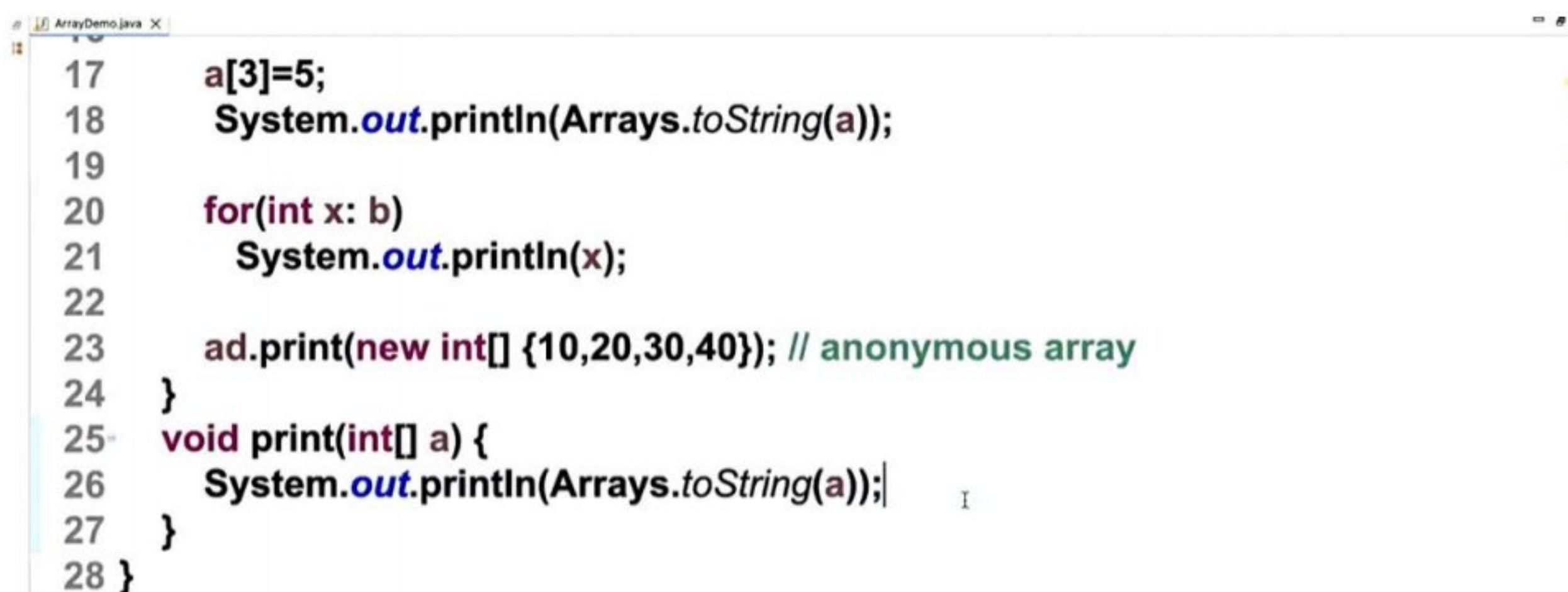
Console Output:

```
<terminated> ArrayDemo (2) [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plugins/org.eclipse.jdt
[0, 0, 0, 0, 0]
0
0
0
0
0
```

Arrays have default initialization even if it is local variable.

8)Anonymous array:

Directly defining the array at the place of creation.



The screenshot shows the Eclipse IDE with a Java file named `ArrayDemo.java` open. The code defines a class `ArrayDemo` with a `main` method. Inside `main`, an integer array `a` of size 5 is created and initialized with default values (0s). The code then prints the array using `Arrays.toString(a)` and iterates over each element using an enhanced for loop, printing each element. The console output on the right shows the array `[0, 0, 0, 0, 0]` and each element `0` on a new line.

```
17     a[3]=5;
18     System.out.println(Arrays.toString(a));
19
20     for(int x: b)
21         System.out.println(x);
22
23     ad.print(new int[] {10,20,30,40}); // anonymous array
24 }
25 void print(int[] a) {
26     System.out.println(Arrays.toString(a));
27 }
28 }
```


9)Is array a collection?

No, array is not a collection.

Reasons:

- 1.Collection are growable in nature. (dynamic)
Arrays are fixed in size. (static)
- 2.Collection can store either homogeneous or heterogenous data
Arrays can store similar data types.
- 3.Collections have rich inbuilt method support.
Arrays doesn't have rich readymade method support.
- 4.Collection can store only objects.
Arrays can store either primitive or objects.
- 5.Every collection is built on top of standard data structure.
Array does not have a standard data structure.

10? Why array index starts at 0?

The index of an array element often corresponds directly to memory address.

Starting from 0 simplifies the calculation of memory address.

For instance, the memory address of the first element is the base address of the array plus 0 times the size of each element, which simplifies to just the base address.

$$\text{arr}[\text{index}] = \text{base reference} + (\text{index} * \text{bytes})$$


```
31 a[0] = base reference (2000)
32 a[0] = 2000 to 2003
33 a[1] = 2004 to 2007
34 a[2] = 2008 (2000 + 2*4)
35
36 a[5] = 2000 + (5*4) = 2020
```

11) What is the output of the code snippet?

```
23 int j[][] = new int[4][];
24 j[0] = new int[]{0,1,3};
25 j[1] = new int[]{6,4};
26 j[2] = new int[]{1,7,6,8,9};
27 j[3] = new int[]{5};
28 System.out.println(j[2][2] == j[1][0]);
29
```

The given array is an example for jagged array.

Jagged array – Each row can have number of columns. (Dissimilar length of elements)

Output: true

12) Cloning of an array:

Creating an identical copy of an object called object cloning.

13) What is the output of code snippet?


```

1 package part1;
2
3 public class Array2Demo {
4     public static void main(String[] args) {
5         int arr[] = {10,20,30};
6         int cloneArr[] = arr.clone();
7         System.out.print(arr==cloneArr);
8         arr[1]=200;
9         System.out.print(arr[1]==cloneArr[1]);
10    }
11 }

```

Output:

```

1 package part1;
2
3 public class Array2Demo {
4     public static void main(String[] args) {
5         int arr[] = {10,20,30};
6         int cloneArr[] = arr.clone();
7         System.out.print(arr==cloneArr);
8         arr[1]=200;
9         System.out.print(arr[1]==cloneArr[1]);
10    }
11 }
12
13

```

```

<terminated> Array2Demo [Java Application] (Applications/Eclipse.app/Contents/Eclipse/plugins/org.eclipse.jdt
falsefalse

```

14)Cloning of one-dimensional array:

```

1 package part1;
2
3 public class Array2Demo {
4     public static void main(String[] args) {
5         int arr[] = {10,20,30};
6         int cloneArr[] = arr.clone();
7         System.out.println(arr==cloneArr);
8         System.out.println(arr);
9         System.out.println(cloneArr);
10        arr[1]=200;
11        System.out.println(arr[1]==cloneArr[1]);
12    }
13 }

```

```

<terminated> Array2Demo [Java Application] (Applications/Eclipse.app/Contents/Eclipse/plugins/org.eclipse.jdt
false
[!@17550481
[!@735f7ae5
false

```

When we clone the one-dimensional array, it will not point to the original object.

In one dimensional array it will do deep copy.

When we create a cloning of one-dimensional array it will create a new object. If we did modification in original array means it does not affect the cloned array.

15)Cloning of two-dimensional array:

Shallow copy happens in two-dimensional cloning.

```
ArrayDemo.java  Array2Demo.java X
1 package part1;
2
3 public class Array2Demo {
4     public static void main(String[] args) {
5         int arr[][]= {{1,2,3,4},{5,6}};
6         int cloneArr[][]=arr.clone();
7         System.out.println(arr);
8         System.out.println(cloneArr);
9         System.out.println(arr[1]==cloneArr[1]);
10        arr[1][1]=200;
11        System.out.println(arr[1][1]==cloneArr[1][1]);
12    }
13 }
```

Output:

```
1 package part1;
2
3 public class Array2Demo {
4     public static void main(String[] args) {
5         int arr[][]= {{1,2,3,4},{5,6}};
6         int cloneArr[][]=arr.clone();
7         System.out.println(arr);
8         System.out.println(cloneArr);
9         System.out.println(arr[0]);
10        System.out.println(cloneArr[0]);
11        System.out.println(arr[1]==cloneArr[1]);
12        arr[1][1]=200;
13        System.out.println(arr[1][1]==cloneArr[1][1]);
14    }
15 }
```

```
<terminated> Array2Demo [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plugins/org.eclipse.justj
[[I@64616ca2
[[I@735f7ae5
[[I@180bc464
[[I@180bc464
true
true
```

Shallow Copy:

Only the outer object is copied. Inner reference-type fields (objects) are shared between the original and the clone.

Deep Copy:

Both the outer object and all inner referenced objects are cloned. So changes in one object do not affect the other.

16)What is the output of the following code snippet upon execution?

```
int[] a=new int[] {2,6,23,44,55};  
System.out.println(Arrays.binarySearch(a, 50));
```

-1
-2
-3
-5

Arrays.binarySearch():

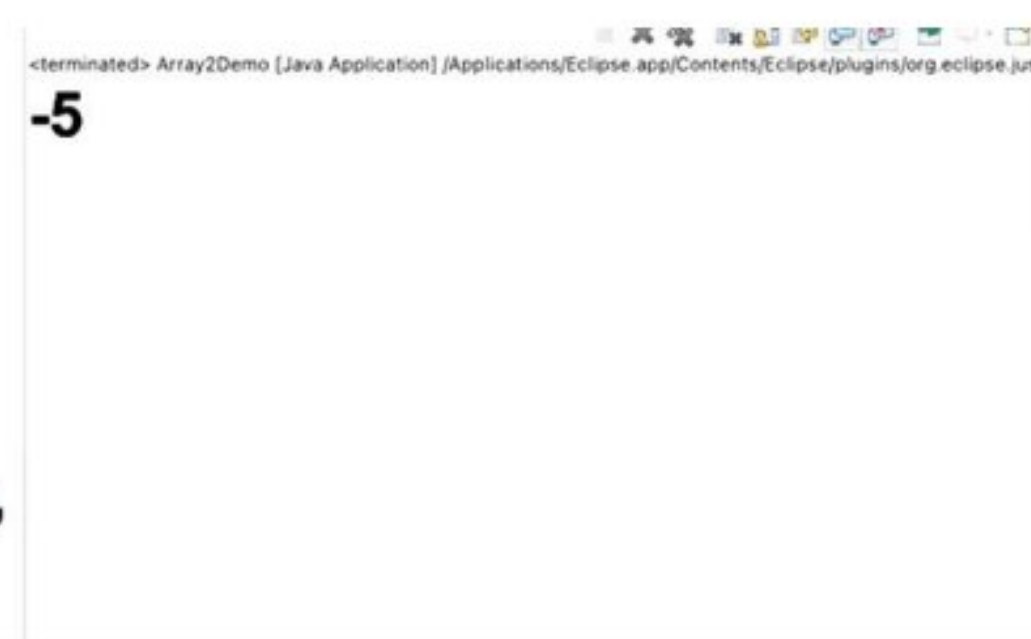
The Arrays.binarySearch() method is part of the java.util.Arrays class and implements the binary search algorithm.

Binary search is an efficient search algorithm that works on sorted arrays by repeatedly dividing the search space in half.

If the key is found, the method returns its index. If the key is not found, it returns a negative value representing the insertion point.

Output:

```
1 package part1;  
2 import java.util.Arrays;  
3  
4 public class Array2Demo {  
5     public static void main(String[] args) {  
6         int[] a=new int[] {2,6,23,44,55};  
7         System.out.println(Arrays.binarySearch(a,  
8     }  
9 }
```



The screenshot shows a Java application window titled "Array2Demo [Java Application]". The window content displays the output of the program, which is "-5". The title bar includes standard window controls and the file path: "C:\Applications\Eclipse.app\Contents\Eclipse\plugins\org.eclipse.jstl".

17)What is the output of the following code snippet upon execution?


```

1 package part1;
2 import java.util.Arrays;
3
4 public class Array2Demo {
5     public static void main(String[] args) {
6         int[] a= {6, -4, 12,0,-10 };
7         int x=12;
8         System.out.println(Arrays.binarySearch(a, x));
9     }
10 }

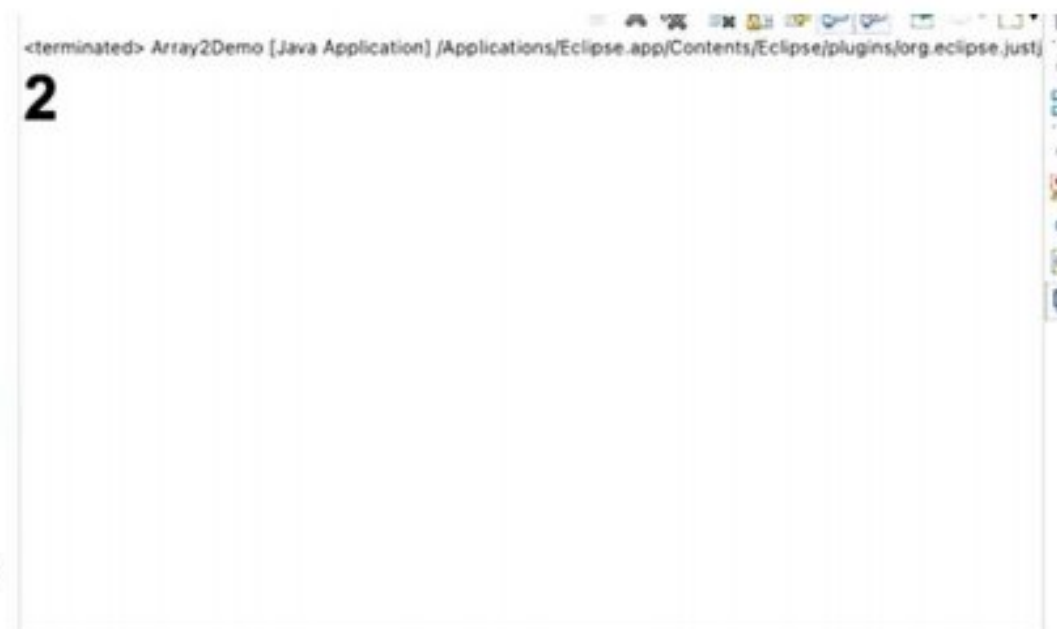
```

Output:

```

1 package part1;
2 import java.util.Arrays;
3
4 public class Array2Demo {
5     public static void main(String[] args) {
6         int[] a= {6, -4, 12,0,-10 };// -10 -4 0 6 12
7         int x=12;
8         System.out.println(Arrays.binarySearch(a,
9     }
10 }

```



The screenshot shows a Java application window titled "Array2Demo [Java Application]". The window contains a single line of text, the number "2", which is the output of the binary search operation performed in the code. The window's title bar includes standard OS controls (minimize, maximize, close) and a status bar at the bottom.